Praxeological Policy Analysis System

Project plan: from documented method to evaluated AI system

Daniel Jin | October 2026 | Working draft, version 2

1. Summary

Policies are usually justified by their visible, intended effects. Their invisible effects, such as the people who respond to new incentives in unplanned ways, the opportunities given up, and the costs that fall on groups nobody was counting, tend to surface only after the damage is done. Mainstream policy analysis is often driven by statistical or formal models, which describe average behavior well but do not reason explicitly about why individuals act as they do.

This project builds a system that analyzes the consequences of a policy using praxeology, the deductive study of purposeful human action developed by Ludwig von Mises and extended by the Austrian School. The system should produce predictions that are both accurate and interpretable: every predicted consequence comes with an explicit causal chain that shows how it follows from the policy and from stated premises.

The work has four phases, run as a loop rather than a straight line:

1. **Document and organize the method.** Turn praxeological reasoning from books and expert practice into an explicit, structured procedure with rules, categories and worked examples.
2. **Implement it with AI.** Combine three kinds of model: Jev, a fast model that returns typed decisions with probabilities; a symbolic reasoning engine, which does formal deduction and keeps the proof; and LLMs, which read policy text, supply context and explain results. Two configurations are built and compared.
3. **Run a theoretical review-refine loop.** Stress-test the method and the implementation on paper and with experts before any real-world data is involved.
4. **Evaluate and feed the results back.** Measure accuracy (F1) and interpretability on toy and historical cases, then use the errors to revise Phases 1 to 3.

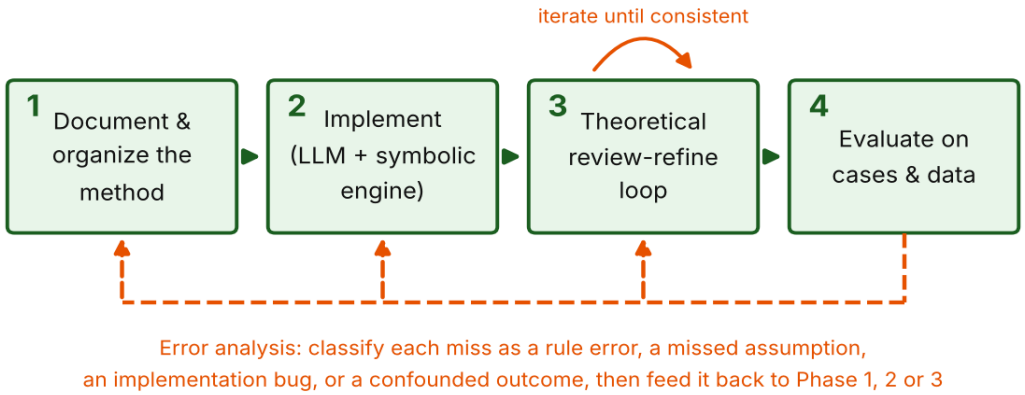


Figure 1. The four phases and their feedback paths.

Plan at a glance

Phase	Main activities	Deliverables	Exit criterion (proposed)
1. Document the method	Collect sources, extract reasoning patterns, define the ontology, rules and analysis workflow, write worked examples	Method Handbook, rule catalog, intervention taxonomy, 25+ worked examples	Experts can reproduce each worked example from the Handbook alone
2. Implement	Build the pipeline in two configurations (Jev beside a symbolic engine, and Jev as a step-by-step decider): parser, typed decisions, engine, verifier, explainer	Two working prototypes, rule base and question schemas in code, test suite, consequence-graph format	Both prototypes reproduce all worked examples end to end
3. Review and refine (theory)	Expert review, consistency and metamorphic tests, red-team cases, derivation audits	Review log, regression suite, versioned rule base	No open critical flaws; audit finds no unsupported steps
4. Evaluate	Run toy and historical cases against baselines, score F1 and interpretability, classify errors	Evaluation report, error taxonomy, prioritized fix list	Beats baselines on held-out cases; fix list drives the next loop

2. Goal and design principles

The goal, in the project's own terms, is to develop a system that uses AI to apply praxeological methods to analyze the consequences of enacting a policy, with predictions that are accurate and interpretable. Six principles follow from that goal and should guide every design decision.

- **Supported, not guessed.** A consequence is only asserted if it comes with a recorded chain: either a formal derivation from stated premises, or a trail of typed decisions that each cleared a confidence threshold. If an LLM suggests something plausible that has neither, it is labelled as a suggestion, not a result.
- **Interpretable by construction.** Every output carries its derivation: the chain of steps, the rule used at each step, and the assumptions the conclusion depends on. This is the opposite of a black box that returns an answer without a reason.
- **Seen and unseen.** Following Bastiat, each consequence is tagged as visible (intended, immediate, concentrated) or invisible (indirect, delayed, diffuse, or an opportunity cost). Surfacing the invisible effects is the main value of the tool.
- **Qualitative and conditional.** Praxeological reasoning supports statements of direction and ordering (more or less of something, who gains and who loses, sooner or later), not precise magnitudes. Conclusions hold under stated conditions, and the system should say so.
- **Value-neutral.** The system describes consequences by group and time horizon. It does not say whether a policy is good. Weighing the consequences is left to the reader, which fits the aim of supporting informed voting rather than prescribing outcomes.
- **Measured against reality.** Claims about the world are tested on cases with known outcomes, and misses are treated as information about the method, not as embarrassments to explain away.

3. Background: what praxeology contributes

Praxeology starts from a single undeniable fact, that humans act: they choose means to pursue ends in a world of scarcity and time. From that action axiom, together with a few supporting premises, it deduces general propositions. Mises set out the system in Human Action (1949); Rothbard extended it into a full treatment of economics and of government intervention in Man, Economy, and State and Power and Market.

Core ideas the Method Handbook will need to capture

- **Action, means and ends.** Individuals act to replace a less satisfactory state with a more satisfactory one, using scarce means. Every action has an opportunity cost: the next-best use given up.
- **Subjective value and marginal utility.** Value is ordinal and personal. Choices are made at the margin, which yields the law of diminishing marginal utility and, from it, the downward-sloping demand schedule.
- **Time preference.** Present goods are preferred to future goods, all else equal, which explains saving, investment and interest.
- **Methodological individualism.** Social outcomes are explained by tracing the actions of individuals, not by treating aggregates as acting agents.
- **Prices and calculation.** Market prices carry dispersed information and make economic calculation possible. Interventions that distort prices degrade that signal (the calculation and knowledge problems, associated with Mises and Hayek).
- **Typology of intervention.** Rothbard separated autistic intervention (restricting a person's own actions), binary intervention (a forced exchange between government and an individual, such as taxation) and triangular intervention (forcing or prohibiting exchanges between private parties, such as price controls). Each type has characteristic consequences.
- **The seen and the unseen.** Bastiat's broken-window fallacy and Hazlitt's "one lesson", to look at the effects of a policy on all groups and over the long run, give the system its guiding question.

An honest note on the method's status

Many Austrian economists hold that the core theorems of praxeology are derived a priori and cannot be refuted by data, while mainstream economists question that stance and test predictions empirically. This project does not need to settle the dispute. The practical position taken here is that the axioms and valid deductions are treated as fixed, while the auxiliary assumptions needed to apply them to a real policy (for example, that a law is actually enforced, or that people can substitute to another good) are empirical and can be checked. Evaluation against historical outcomes therefore tests the application of the method and its stated conditions. When a prediction misses, the first question is which assumption failed, not whether the logic itself is wrong.

4. Phase 1: Document and organize the method

Today the method lives in books and in the heads of people who practice it. Phase 1 turns it into something a machine can use and a newcomer can follow: an explicit, structured, tested procedure. This is also where most of the project's quality will be decided, because the system can only be as sound as the method it encodes.

4.1 Build the source corpus

- Primary texts: Mises, Human Action; Rothbard, Man, Economy, and State and Power and Market; Hazlitt, Economics in One Lesson; Bastiat, "That Which Is Seen, and That Which Is Not Seen"; Hayek, "The Use of Knowledge in Society"; Mises on economic calculation; Hoppe on the Austrian method.
- Applied case writing: textbook and article treatments of price controls, taxes, subsidies, tariffs, licensing, prohibition and monetary policy.
- Expert practice: recorded walkthroughs of how praxeologists analyze a specific policy step by step, including the "obvious" moves they normally skip. These are the most valuable and hardest to find, so schedule them early.
- Counter-literature: mainstream and behavioral critiques of the same policies, kept as a reference set so the method's limits are known from the start.

4.2 Define the ontology

Agree on a small, precise vocabulary before writing any rules. A first draft:

Concept	Definition used in the system
Actor class	A group defined by role in the situation (for example buyers, sellers, landlords, tenants, taxpayers, officials), not by individual identity.
End and means	What an actor class is trying to achieve, and the scarce resources it can use.
Good / variable	A thing or quantity that can rise or fall: price, quantity supplied, quality, wait time, investment, savings.
Intervention	A coercive change to the rules of the game, typed as autistic, binary or triangular, with a precise statement of what is required, prohibited or transferred.
Direct effect	The immediate, intended change the policy produces (the visible part).
Response	A change in an actor class's choices caused by a changed incentive or constraint.
Consequence	A normal-form claim: actor class, variable, direction, time horizon, condition.
Assumption	A premise beyond the action axiom, labelled with its source and whether it is empirical.
Visibility	Visible (intended, immediate, concentrated) or invisible (indirect, delayed, diffuse, or a forgone alternative).

4.3 Write the analysis workflow

Turn the experts' reasoning into a repeatable sequence of steps. A first draft of the workflow, to be revised with experts:

#	Step	Question answered and output
1	State the policy precisely	What exactly is required, prohibited, taxed, subsidized or transferred, by whom, and for whom? Output: a structured policy record.

#	Step	Question answered and output
2	Classify the intervention	Is it autistic, binary or triangular? Which characteristic consequences does that type imply?
3	Identify actor classes	Who is directly bound, who benefits, who pays, who is a bystander whose options change?
4	Identify the direct effect	What is the immediate, intended change? This is the visible consequence.
5	Trace responses	For each actor class, how do its constraints and incentives change, and what will it do differently at the margin (substitute, relocate, evade, lobby, stop producing)?
6	Trace second-order effects	How do those responses affect other classes and markets? Repeat until effects become negligible or the chain exceeds a set depth.
7	Add time structure	Which effects arrive quickly and which accumulate (capital consumption, expectations, institutional drift)?
8	Identify opportunity costs	What resources were diverted, and what would have been done with them? This is the main source of invisible consequences.
9	State conditions	Which assumptions does each conclusion depend on (enforcement, elasticity, availability of substitutes)? What would reverse it?
10	Report	Consequences by actor class and horizon, split visible and invisible, each with its derivation and assumptions.

4.4 Encode reasoning as rules

Each reusable inference becomes a named rule with an explicit pattern: preconditions, conclusion, the assumptions it needs, and a pointer to its source. Keeping rules small and separate makes them easy to review and combine. The first catalog should cover the main intervention types: price ceilings and floors, taxes and subsidies, quotas and tariffs, licensing and entry barriers, prohibition, and monetary or credit expansion.

A worked example of how a rule and its premises might look, with illustrative syntax:

```
% Rule PR-017: binding price ceiling -> shortage
% Premises: law of marginal utility (axiom-derived); ceiling is enforced
(empirical)
derive(shortage(Good, Market), rule(pr_017),
       assumes([enforced(Cap, Market)])) :-
    price_ceiling(Good, Market, Cap),
    market_clearing_price(Good, Market, P),
    Cap < P.
```

Each rule can also be written as a set of narrow typed questions. This lets the same Handbook drive both implementation configurations in Phase 2. For rule PR-017:

```
# Typed-question form of PR-017 (illustrative)
noul:   "Does the policy set a maximum price for the good?"
noul:   "Is that maximum below the price the market would clear at?"
noul:   "Is the maximum likely to be enforced?"
choice: "Which actor class is directly bound?" [sellers, buyers, other]
score:  "How soon will the supply response appear?" [short, medium, long]
```

And the structured record a single derived consequence could produce:

```
{
  "consequence": "Quantity of rental housing offered falls over time",
  "actor_class": "landlords",
  "variable": "rental units available",
  "direction": "decrease",
  "horizon": "medium to long run",
  "visibility": "invisible",
  "tier": "B",
  "derivation": ["PR-017 rent cap", "IN-004 lower return",
                 "SU-002 reduced supply"],
  "assumptions": ["cap is binding and enforced",
                  "owners can withdraw units"]
}
```

Every consequence is labelled with a tier, which is central to how results are later scored and trusted:

Tier	Meaning	Example of what belongs here
A	Derived from the action axiom and axiom-level laws alone	A good bought at a forced lower price is demanded in a larger quantity than before.
B	Derived with named empirical assumptions	A binding rent cap, if enforced and if owners can withdraw units, reduces rental supply over time.
J	Supported by a trail of typed decisions above the confidence threshold, with no formal proof (Jev step-by-step configuration)	A step judged “the cap is binding” at high probability, with each answer and its confidence logged.
C	Suggested by the LLM or by analogy, not derived	A plausible side effect with no rule yet. Reported separately and never mixed into Tier A or B counts.

4.5 Worked examples and the first test set

- Write at least 25 complete worked analyses by hand with the experts, from textbook-simple (broken window, price ceiling on one good) to messy (a multi-part bill).
- Each worked example records the policy, the full analysis, the rules used, and the assumptions. These become the first regression tests and few-shot examples.

- Mark any step where the experts relied on a move that is not yet in the rule catalog. Those gaps are the to-do list for the catalog.

> **Phase 1 exit test.** Give a new person the Handbook and three unseen policies. If they can produce analyses that experts accept, the method is documented well enough to encode.

5. Phase 2: Implement the system with AI

The central design choice is to split the work by what each kind of component does well. Three kinds are used:

- **Jev (typed decisions).** A model from TypeSafe AI that does not write text. It receives a state and a set of typed questions and returns an answer to each with a confidence value. The types are Noul (yes or no, as a probability), Choice (pick one option) and Score (rate on a rubric). It is fast (70 to 500 ms quoted), cheap, and constrained to valid answer types.
- **Symbolic engine.** Reliable at deduction and able to show a formal proof, but it cannot read a legislative text.
- **LLM.** Good at reading messy language, drawing on background knowledge and writing explanations, but it can invent steps, and its stated reasoning is not always its real reason.

Jev cannot explain its answers, and its documentation lists counting, arithmetic and date comparison as unreliable, so it is used for bounded judgments. Anything that needs a proof, a calculation or prose goes elsewhere. Its speed and cost figures are vendor-reported and its calibration still needs checking on this project's own questions, which is built into Phases 3 and 4.

5.1 Two configurations

Both configurations share the same Handbook, rule catalog and evaluation set. They differ in where the reasoning happens, and Phase 4 compares them directly.

Aspect	Configuration A: Jev beside the engine	Configuration B: Jev as step-by-step decider
Where reasoning happens	The symbolic engine forward-chains formal rules over facts.	A loop of typed questions that Jev answers at each step.
What Jev does	Classifies and supplies facts: intervention type, actor classes, and the probability that each assumption holds.	Decides each step: which rule pattern applies, whether preconditions hold, whether further effects follow, and their horizon and visibility.
Interpretability	A formal derivation tree with assumptions (Tier A or B).	A decision trail of questions, answers, probabilities and confidence (Tier J). Readable, but not a proof.
Uncertainty	Enters as assumption probabilities attached to conclusions.	Enters at every step; confidence can be combined along the chain.
Strengths	Strongest guarantees; deduction is deterministic and checkable.	Simpler to build and extend; needs only question schemas, not full formal encoding; very fast.

Aspect	Configuration A: Jev beside the engine	Configuration B: Jev as step-by-step decider
Weaknesses	More upfront encoding effort; rules must be formal.	Errors can compound across steps; weaker audit; depends on question quality and calibration.
Best used for	Core rules that can be formalized, and high-stakes results.	Quick coverage of new intervention types, and drafting rules to formalize later.

Configuration A: Jev beside the symbolic engine

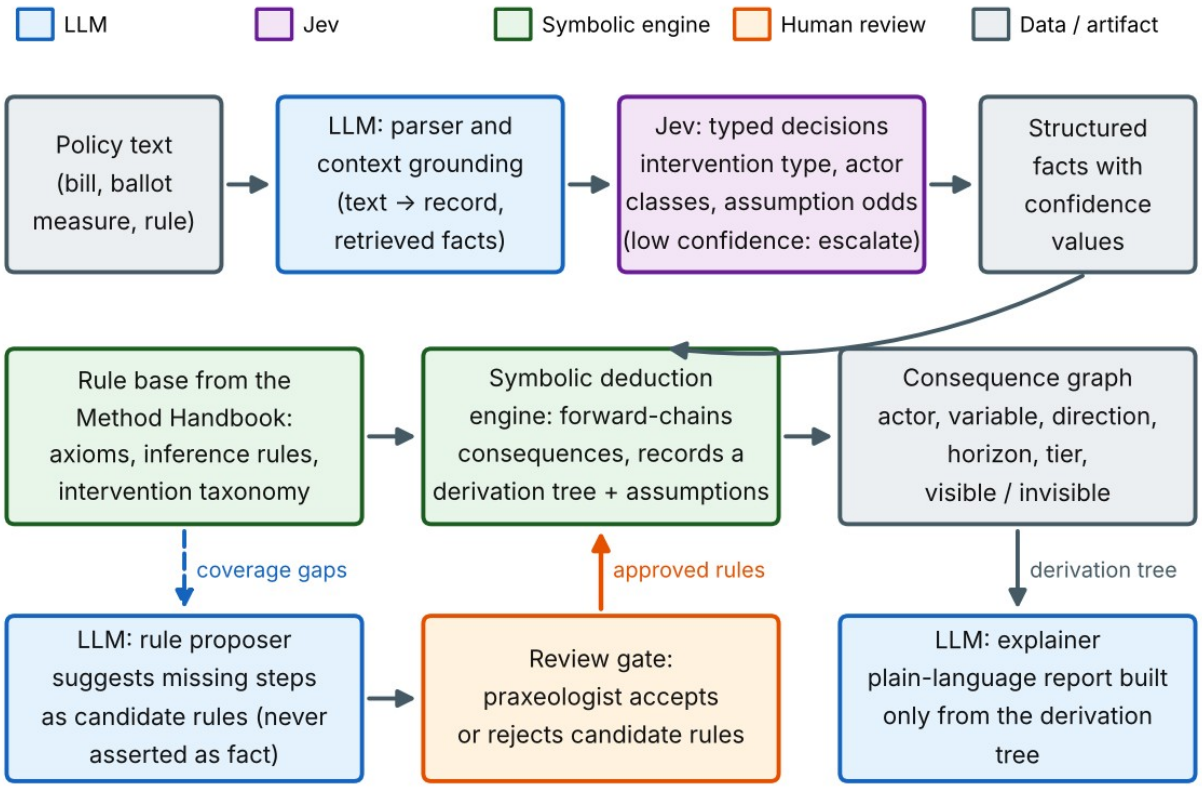


Figure 2. Configuration A. Reasoning happens in the symbolic engine; Jev supplies typed facts; LLMs handle language, context and explanation.

Configuration B: Jev as the step-by-step decider

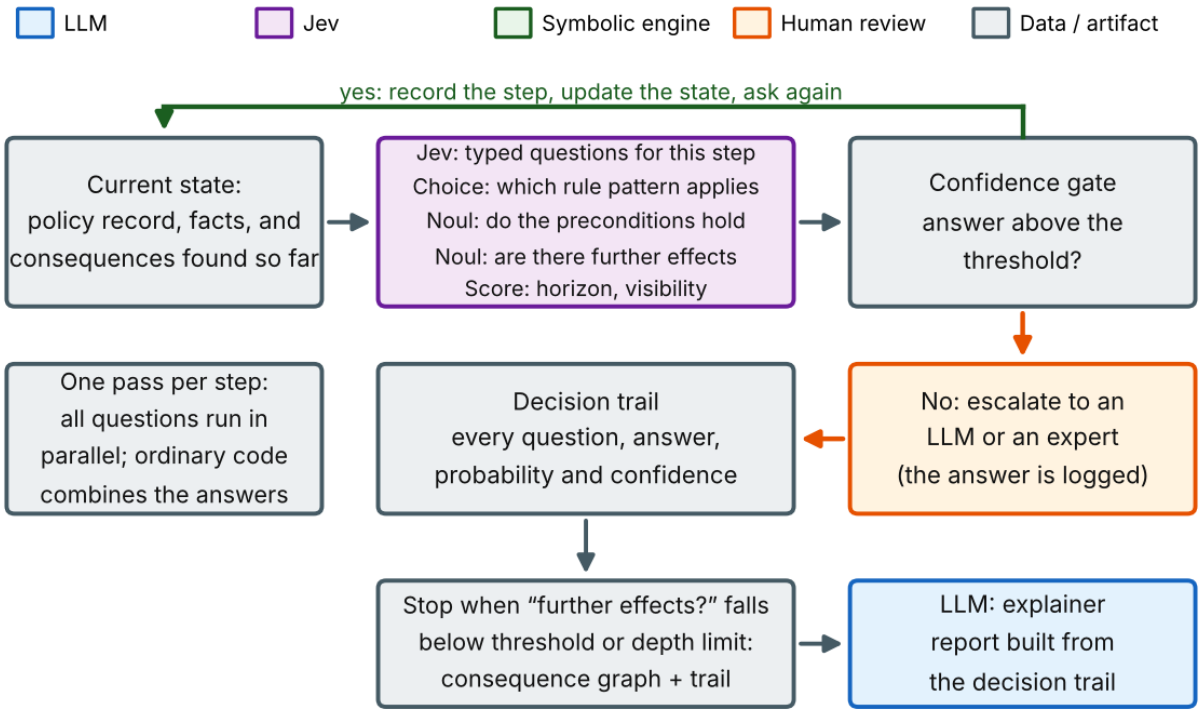


Figure 3. Configuration B. Each step is a set of typed questions; confident answers extend the chain and uncertain ones escalate.

A hybrid is also possible: use Configuration B to explore a new intervention type quickly, then promote the decisions experts keep into formal rules for Configuration A. The tier on each result records which path it took.

5.2 Components and their roles

Component	Job	Guardrail
LLM: policy parser	Convert policy text into a structured record (who, what, when, penalties, exemptions) matching a fixed schema.	Schema-constrained output; every field cites the source sentence; ambiguous fields are flagged, not guessed.
LLM: context grounding	Retrieve facts about the market, existing law and past experience that the rules need as inputs.	Facts carry sources and are stored as inputs, kept separate from derived claims.
Jev: typed decisions	Answer narrow typed questions about the policy record: intervention type (Choice), actor classes (Choice), whether an assumption holds (Noul), horizon and visibility (Score). In Configuration B, also drives each reasoning step.	One concept per question; a confidence threshold per question; below threshold, escalate to an LLM or expert; counting, arithmetic and dates stay in code.

Component	Job	Guardrail
Rule base	The encoded axioms, inference rules and intervention taxonomy from the Method Handbook, plus the typed-question form of each rule.	Versioned in source control; every rule has an ID, a source, tests, a reviewed question schema and a named reviewer.
Symbolic engine (A)	Forward-chain from the facts and rules to a set of consequences, recording the derivation tree and the assumptions used.	Deterministic: the same input gives the same output. Detects contradictory rules.
LLM: rule proposer	When the engine stalls or an expert notes a gap, suggest candidate rules, question schemas or missing steps.	Output is a candidate only. It enters the rule base only after expert approval and new tests.
Verifier	Check that each step is a valid instance of a rule (A), or that each decision in the trail cleared its threshold and has a logged question and answer (B).	Rejects any consequence without a complete derivation or trail, or an explicit Tier C label.
LLM: explainer	Write the plain-language report from the derivation tree or decision trail, in terms a non-expert can follow.	Receives only the tree or trail. A check compares the text to it for added or missing claims.

5.3 Choices to make

- **Jev access and versions.** Jev is in early access through a waitlist at console.typesafe.ai, and is also offered through Vercel AI Gateway, Cloudflare Workers AI (as typesafe/jev) and a LangChain integration. Pin an exact version (for example jev-1.13.0) rather than a moving alias, and log the version on every run so results can be reproduced.
- **Question design.** Keep each question narrow and combine the answers in code, as TypeSafe recommends. Write questions with the experts, test them for ambiguity, and keep one schema file per rule.
- **Thresholds and escalation.** Set a confidence threshold per question type from calibration data, not by guesswork. Below threshold, escalate to an LLM or expert and log it. TypeSafe also provides an adapter that runs existing LLMs against the same question schema, which is useful for benchmarking Jev against LLM-only answers.
- **Input size.** Jev has a 32,000-token context window and loses accuracy on large, noisy state, so pass compact structured records rather than full legislative text.
- **Symbolic engine.** Options include Datalog or Prolog (easy to read, good at forward chaining and provenance), answer set programming such as clingo (handles defaults and exceptions well), or a proof assistant such as Lean (strongest guarantees, steepest effort). A reasonable path: start with Datalog or Prolog for speed, keep rules in a neutral format, and move the Tier A core to a stricter system later if it is worth the cost.
- **Rule representation.** Keep rules at the level of qualitative relations (increases, decreases, substitutes for, is a cost of) rather than numeric equations, matching the method's own level of precision.

- **How much freedom the LLM gets.** Start tightly constrained (parse, ground, explain) and widen only as evaluation shows the proposer helps without adding unsupported claims.
- **Software stack.** Python with typed data models for the policy record, decision trail and consequence graph, a graph library, a standard testing framework, source control with tagged rule-base releases, and an experiment log recording prompt, model and version, rule-base version and output for every run.
- **Model usage.** Use deterministic settings where possible and keep prompts and question schemas in files under version control.

5.4 Build order

1. Encode three or four rules and one worked example both ways: as formal rules for the symbolic engine and as typed-question schemas for Jev. Confirm that both outputs are readable.
2. Add the LLM parser and the structured policy schema so the same example runs from raw text.
3. Stand up Jev with a small set of questions and measure its calibration on the Phase 1 worked examples before relying on it.
4. Grow the catalog intervention type by intervention type, adding formal rules, question schemas and tests together.
5. Add context grounding, then the verifier, then the explainer.
6. Add the rule proposer last, behind the review gate, once the base pipeline is stable.

> **Phase 2 exit test.** Both configurations reproduce every Phase 1 worked example from raw policy text, with derivations or decision trails that match the experts' reasoning.

6. Phase 3: Theoretical review-refine loop

Before testing against real-world data, test the method on paper. This loop is cheap, fast and catches a different class of error than evaluation does: contradictions, gaps, unstated assumptions and drift between what the experts meant and what the system does.

6.1 Review activities

Activity	What it checks
Expert derivation review	Experts read a sample of derivations and rate each step as valid, needs a stated assumption, or invalid. Disagreements go back to the Handbook.
Rule-base consistency checks	Automated search for rules that contradict each other, loop, or give different answers for the same situation.
Metamorphic tests	Rephrase the same policy and check that the output is unchanged. Reverse a policy (repeal a ceiling) and check that effects reverse. Scale up a parameter and check that direction holds.
Red-team cases	Hand-built situations designed to break the method: policies with mixed instruments, exemptions that change incentives, or cases where an assumption plainly fails.
Assumption audit	Every conclusion lists its assumptions; reviewers ask what is missing and what would reverse the result.

Activity	What it checks
Coverage review	Compare the rule catalog to the intervention taxonomy to find policy types with no rules.
Layperson readability test	Have someone with no background follow an explanation and restate it. Confusion points mean steps are missing or jargon is leaking through.
Question-schema review	Experts read each typed question for ambiguity, double-barreled wording and leading phrasing. One concept per question.
Calibration check	Run Jev on the worked examples and check that confidence tracks accuracy (for example, answers at 90% confidence are right about 90% of the time). Escalation thresholds come from this.
Ablation checks	Remove one rule and see what changes. Rules that change nothing are redundant; rules whose absence breaks many cases are critical and deserve extra scrutiny.

6.2 Running the loop

1. Run all review activities on the current rule-base version and log every finding with a severity (critical, major, minor).
2. Fix findings in the Handbook first, then in the rule base, then in code, so the documentation never lags the implementation.
3. Add a regression test for every fixed finding.
4. Re-run the full regression suite, then repeat until no critical or major findings remain open.
5. Tag a release of the rule base and freeze it for evaluation.

> **Phase 3 exit test (proposed).** No open critical findings; a random audit of derivations finds zero unsupported steps; paraphrase tests give the same consequences in at least 95% of cases; every rule has a source, a test, a reviewer and a reviewed question schema.

7. Phase 4: Evaluate and refine

Phase 4 asks whether the system works on the world, not just on the reviewers' own examples. It is designed around the two properties named in the goal: accurate and interpretable.

7.1 Evaluation sets

Set	Contents	Purpose
Toy cases (about 30)	Classic textbook situations: broken window, single-good price ceiling and floor, a simple tax or subsidy, import tariff, a ban on a good, the "cobra effect" type of perverse incentive.	Check that the method gets the fundamentals right, and that behavior is stable. Cheap to run many times.

Set	Contents	Purpose
Historical policies, development split (about 40)	Well-documented past policies with studied outcomes, such as rent control expansions, wartime and 1970s wage and price controls, Prohibition, major tariff episodes, minimum-wage changes, occupational licensing, and vehicle trade-in subsidy programs.	Day-to-day error analysis and refinement.
Historical policies, held-out test (about 15 to 20)	A sealed set of similar cases that nobody looks at while developing.	An honest final score. Opened only at milestones to avoid fitting the method to it.
Forward-looking log	Predictions about current or pending policies, recorded with a timestamp before the outcome is known.	A genuine out-of-sample test, evaluated later. This is the "inference" mode.

7.2 Ground truth: outcome cards

For each historical case, create an outcome card listing the observed consequences with their sources, written by one person and checked by another. Prefer peer-reviewed studies and official data. Record the actor class, variable, direction and horizon in the same normal form the system uses, and note any major confounders (concurrent events that could explain the outcome). Where the evidence is disputed, record both findings and treat the case as lower-confidence in scoring.

> **Contamination risk.** An LLM may already know how famous policies turned out, which could leak the answer into its "prediction". Mitigate by prompting with the situation as it stood at enactment, by measuring against an LLM-only baseline that has the same knowledge, by weighting the held-out and forward-looking sets more heavily, and by including less-famous cases.

7.3 Accuracy: F1 on consequences

Each predicted consequence is compared with the observed consequences. A prediction matches an observation when the actor class (or a clearly overlapping class), the variable and the direction agree and the time horizons overlap. Then:

- **Precision** = matched predictions / all verifiable predictions. It penalizes consequences that were predicted but did not occur.
- **Recall** = matched observed consequences / all observed consequences. It penalizes consequences that occurred but were not predicted.
- **F1** = the harmonic mean of the two, so the system cannot score well by predicting everything or almost nothing.

Several refinements are needed because real outcome data is incomplete:

- **Unverifiable predictions.** If a predicted effect was never studied, it is neither confirmed nor refuted. The primary score excludes these from precision and reports their rate separately; a sensitivity analysis counts them as misses.
- **Slices.** Report F1 by tier (A, B, J, C), by visibility (visible versus invisible), by intervention type and by time horizon. A system that only finds visible effects would show up here immediately. Recall on invisible consequences is the headline metric for this project's purpose.

- **Matching procedure.** Normalize both sides to the same form, propose matches with an automated matcher (embedding similarity plus an LLM judge), and have a person adjudicate a sample. Report agreement between the matcher and the human, and between two human annotators.
- **Uncertainty.** Give confidence intervals by bootstrapping over cases, and use paired comparisons against baselines on the same cases.

7.4 Interpretability

Measure	How it is assessed	Target (proposed)
Derivation validity	Automated check that each step is a valid instance of a rule with listed assumptions, plus expert audit of a sample.	100% of Tier A and B results; experts accept at least 90% of audited steps
Explanation faithfulness	Compare the written explanation to the derivation tree; flag claims in the text that are not in the tree, and steps in the tree that are missing from the text.	Fewer than 2% of claims unsupported
Decision-trail validity (Configuration B)	Experts rate each logged question and answer as correct, plausible but wrong, or wrong, and check that every step cleared its threshold.	At least 90% of audited decisions correct
Expert acceptance	Praxeologists rate whether they would endorse the chain on a 1 to 5 scale.	Median of 4 or higher
Layperson comprehension	Non-experts read a report, explain the chain back, and answer questions about it.	Most readers restate the main chain correctly
Assumption transparency	Share of conclusions that list their assumptions and the conditions that would reverse them.	100%

7.5 Comparing configurations and checking calibration

Phase 4 also answers the design questions left open in Phase 2. Run these systems on the same cases:

- Configuration A (Jev beside the symbolic engine).
- Configuration B (Jev as step-by-step decider).
- Each configuration with an LLM in place of Jev, to show what Jev adds beyond a general model.
- Jev alone, answering a fixed question set, as a floor for how much work the questions themselves do.

For each, report F1 and the interpretability measures, plus cost, latency and the escalation rate (the share of decisions below threshold). Calibration is checked separately:

- **Reliability.** Compare stated confidence with actual accuracy in bins, and summarize as expected calibration error. Check Noul, Choice and Score questions separately.

- **Coverage versus accuracy.** Plot accuracy against the share of decisions made above a threshold, to choose thresholds that trade escalation workload against error.
- **Error propagation (Configuration B).** Measure how often an early wrong decision leads to a wrong final consequence, and whether the trail's confidence flags it.
- **Robustness.** Re-run with reworded questions and with irrelevant detail added to the state.

A hybrid (Configuration B to explore, A to formalize) is judged by the same measures.

7.6 Baselines

A score only means something relative to alternatives. Compare against:

- An LLM asked directly, with no framework, to predict the consequences of the policy.
- The same LLM with step-by-step prompting and with an explicit praxeology prompt, to separate the effect of the knowledge from the effect of the symbolic structure.
- The same LLM prompted to give a mainstream economist's analysis, as a reference for a different approach.
- Human analysts: a praxeologist, and a general policy analyst, each working from the same policy text.

If the hybrid system does not beat the strongest LLM-only baseline on accuracy, it should at least match it while being clearly more interpretable. Either result is informative, and the plan should be ready to report either honestly.

7.7 Error analysis and the refine loop

Every miss (a missed or false consequence) is traced and assigned a cause:

Cause	Meaning	Where the fix goes
Rule error	The rule itself is wrong or too broad.	Phase 1 (Handbook) and rule base
Missing rule	No rule covers the needed step.	Phase 1, then encode in Phase 2
Missed assumption	The rule is fine but its condition did not hold in this case (for example, the law was not enforced).	Phase 1: add the condition and a check; Phase 2: ground the fact
Grounding or parsing error	The LLM misread the policy or retrieved the wrong fact.	Phase 2: prompts, schema, retrieval
Wrong or overconfident decision	A typed answer was wrong, or its confidence was too high.	Phase 2: rewrite the question, adjust the threshold, add escalation. Phase 3: recheck calibration
Confounded outcome	Something else explains the observed result.	Annotate the case; do not change the method
Unmeasured or disputed outcome	The evidence is missing or contested.	Annotate; exclude or down-weight

Fix causes in order of how many cases they affect, add a regression test for each fix, re-run the development set, and repeat. Open the held-out test set only at planned milestones, so that it remains an honest measure. Stop when the system stops improving on the development set, when the remaining errors are mostly confounded or unmeasured, or when the success criteria in Section 9 are met.

8. Timeline

A first estimate in weeks from kickoff. Phases overlap, since encoding can begin on the first finished rules while documentation continues. Adjust once the team and time available are known.

Phase	Weeks	Key milestones
Plan and source gathering	1 to 2	Corpus assembled; expert sessions scheduled; Jev early-access request submitted; engine and schema choices made
1. Document the method	2 to 6	Ontology agreed (wk 3); workflow draft (wk 4); rule catalog v1 and 25 worked examples (wk 6)
2. Implement	5 to 11	Jev access and question schemas started (wk 5); prototypes of both configurations on three examples (wk 7); catalog encoded and calibrated (wk 10); explainer and verifier done (wk 11)
3. Theoretical review-refine	10 to 13	First full review round (wk 11); consistency and metamorphic suite (wk 12); frozen release (wk 13)
4. Evaluate	13 to 17	Outcome cards complete (wk 14); baselines and configuration comparison run (wk 15); development-set results, calibration report and error taxonomy (wk 16); held-out test (wk 17)
Refine and repeat	17 onward	Fix cycles of two to three weeks; forward-looking log started and checked as outcomes arrive

9. Success criteria

Proposed targets, to be revised once baselines show what is realistic:

- Held-out F1 above the strongest LLM-only baseline, with a bootstrap interval that does not overlap zero difference, or equal accuracy with clearly better interpretability scores.
- Higher recall on invisible consequences than every baseline.
- Every Tier A and B result has a complete, valid derivation with listed assumptions, and every Tier J result has a complete decision trail.
- Jev's confidence is calibrated well enough to set useful thresholds, with accuracy and escalation rate reported for each configuration.
- At least 90% of audited steps accepted by experts, and stable output under paraphrase.
- A written error analysis explaining every systematic miss in the held-out set.

10. Risks and mitigations

Risk	Why it matters	Mitigation
Method is under-specified	Experts' reasoning contains tacit steps, so the rules encode a partial picture.	Record expert walkthroughs; ask a non-expert to follow them; track every skipped step as a gap.
LLM invents steps	Unsupported claims erode trust in the whole system.	Deduction only in the symbolic engine or a logged decision trail; tier labels; verifier; proposer behind an expert gate.
Jev is confidently wrong	Valid-looking typed answers can still be wrong, and a probability still has to be acted on.	Calibration checks; per-question thresholds; escalation; accuracy reported at each threshold.
Errors compound across steps	In Configuration B, an early wrong decision can carry through the chain.	Log confidence at each step; escalate low-confidence steps; measure error propagation; promote key steps to formal rules.
Vendor dependence	Jev is a new, early-access model with vendor-reported performance and no public rate limits or tiers.	Pin versions; log every call; keep question schemas portable; use the LLM adapter as a fallback and benchmark.
Over-claiming certainty	Qualitative, conditional conclusions can be read as forecasts.	Always display assumptions and reversal conditions; avoid numbers the method cannot support.
Weak ground truth	Outcomes are confounded, partial, or disputed.	Outcome cards with sources and confounders; unverifiable bucket; sensitivity analysis.
Knowledge leakage	The LLM may recall the actual outcomes.	Enactment-date prompting, LLM-only baseline, sealed and forward-looking sets.
Overfitting to the test cases	Repeated tuning on the same cases inflates scores.	Development and held-out split; open the test set rarely; forward-looking log.
Perceived bias	A single-school method may be seen as advocacy.	Value-neutral reporting; comparison with mainstream baseline; publish assumptions and errors openly.
Scope creep	Trying to cover all policy areas at once.	Start with a few intervention types; widen only when metrics hold.

11. Open decisions

1. Which symbolic engine to start with for Configuration A (Datalog or Prolog, answer set programming, or a proof assistant), and whether to build A and B in parallel or one after the other.

2. How formal the axiom layer should be, and whether disagreements within the Austrian tradition should be modelled as selectable options.
3. How much freedom the LLM gets in the first release, and the review process for rules it proposes.
4. Jev access: early-access approval, the API route (direct or through a gateway), and a fallback if access is limited.
5. Confidence thresholds for Jev decisions, how they are set from calibration data, and who handles escalations.
6. Which historical cases go into development and which into the sealed test set, and who writes and checks the outcome cards.
7. How to score unverifiable predictions in the primary metric.
8. Who the intended first users are (policymakers, voter-information groups, students) and what the report should look like for them.
9. Whether and when to release results publicly, and in what form.

12. After the system works

Once the method is reliable, the system can support informed voting: ballot measures are usually summarized by their visible aims, so a short, plain-language analysis of invisible and delayed consequences, with assumptions listed, would fill a gap. Policy drafters could use it as an early check, and civic groups could use it as a research aid. These uses depend on the reliability established in Phase 4, and on keeping the output descriptive rather than advocating a position.

13. Suggested reading

Praxeology and Austrian economics

- Mises, L. von. Human Action: A Treatise on Economics (1949).
- Rothbard, M. Man, Economy, and State (1962) and Power and Market (1970).
- Hazlitt, H. Economics in One Lesson (1946).
- Bastiat, F. “That Which Is Seen, and That Which Is Not Seen” (1850).
- Hayek, F. A. “The Use of Knowledge in Society” (1945).
- Mises, L. von. “Economic Calculation in the Socialist Commonwealth” (1920).
- Hoppe, H.-H. Economic Science and the Austrian Method (1995).

Neuro-symbolic reasoning and faithful explanation

- Garcez, A. d’A. and Lamb, L. Neurosymbolic AI: The 3rd Wave (2020).
- Pan, L. et al. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning (2023).
- Olausson, T. et al. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers (2023).
- Kambhampati, S. et al. LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks (2024).
- Turpin, M. et al. Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting (2023).

- Lanham, T. et al. Measuring Faithfulness in Chain-of-Thought Reasoning (2023).

Typed decisions and calibration

- InfoQ. TypeSafe AI Jev released (October 2026).
- eesel. TypeSafe Jev review: the “System One” model that gives AI the properties of code (2026).
- Let’s Data Science. TypeSafe AI Launches Jev for Structured Software Decisions (2026).
- TypeSafe AI. Jev documentation and quickstart, via console.typesafe.ai (read October 2026).
- Guo, C. et al. On Calibration of Modern Neural Networks (2017).

Causal reasoning and policy evidence

- Pearl, J. Causality: Models, Reasoning, and Inference (2nd ed., 2009).
- Diamond, R., McQuade, T. and Qian, F. The Effects of Rent Control Expansion on Tenants, Landlords, and Inequality: Evidence from San Francisco. American Economic Review (2019).

The praxeology, neuro-symbolic and policy references were written from memory and should be checked before citing. The Jev sources are web articles read in October 2026; their performance figures are vendor-reported.